Supplementary file on the paper entitled "On Solving the Variable Gapped Longest Common Subsequence Problem"

Marko Djukanović[1,2,6], Nikola Balaban[2], Christian Blum[3], Aleksandar Kartelj[4], Sašo Džeroski[5], and Žiga Zebec[6]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Faculty of Natural Sciences and Mathematics, University of Banja Luka, Banja Luka, Bosnia and Herzegovina
nikola.balaban@student.pmf.unibl.org, marko.djukanovic@pmf.unibl.org
[3] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iia.csic.es
[4] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[5] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[6] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

A ILP model for the fixed $m = 2$ VGLCS Problem

For the case of the VGLCS problem with $m = 2$, in this section, we design an *integer programming model*, representing a methodological approach that differs from most existing approaches in the literature, which are predominantly based on dynamic programming (DP). This is a proof-of-concept showing the structural difficulty of this approach and a baseline modeling contribution when solving the fixed version of the tackled problem. Shortly, the purpose of the ILP model is not computational competitiveness but structural modeling and benchmarking.

To this end, let:

- $M = \{(i, j) \mid s_1[i] = s_2[j]\}$ denotes the set of all positions where the characters match.

For each $(i, j) \in M$, we define a binary variable:

$$x_{i,j} = \begin{cases} 1, & \text{if the pair } (i,j) \text{ is selected as part of the solution,} \\ 0, & \text{otherwise.} \end{cases}$$

We introduce additional binary variables $s_{i,j}$ indicating whether the pair (i, j) represents the *starting position of the subsequence*.

Objective function. We maximize the number of selected admissible matches:

$$\max \sum_{(i,j)\in M} x_{i,j}$$

Constraints. The following constraints are imposed:

1. Predecessors (variable gap). For each $(i,j) \in M$, we define the set of valid predecessors:

   $$\texttt{Pred}(i,j) = \{(i',j') \in M \mid i' < i,\ j' < j,\ i-i' \leq G_{s_1}[i]+1,\ j-j' \leq G_{s_2}[j]+1\}$$

   The activation constraint is given by:

   $$x_{i,j} \leq \sum_{(i',j')\in \mathrm{Pred}(i,j)} x_{i',j'} + s_{i,j}$$

2. *Starting position.* At most one starting pair is allowed:

   $$\sum_{(i,j)\in M} s_{i,j} \leq 1$$

3. *Conflict constraints (character ordering).* Two distinct pairs (i,j) and (i',j') are in conflict if they violate the character order:

   $$\text{If } (i \leq i' \text{ and } j \geq j') \text{ or } (i \geq i' \text{ and } j \leq j'), \text{ then: } \quad x_{i,j} + x_{i',j'} \leq 1$$

4. *Auxiliary constraints*: $s_{i,j} = 1 \Rightarrow x_{i',j'} = 0, \forall (i' \leq i-1, j' \leq j-1)$. The constraints are expressed in the following form:

   $$\sum_{(i',j')|i'\leq i \wedge j' \leq j} x_{i'j'} + s_{i,j} \leq 1, \forall (i,j) \in M$$

Domains of the variables are given by

$$x_{i,j},\ s_{i,j} \in \{0,1\}, \quad \forall (i,j) \in M.$$

The model is formulated to maximize the number of valid matches forming a common subsequence, while respecting the ordering of elements and the maximum allowed gaps between consecutive elements in both sequences.

B Pseudocodes of the IMSBS approach

Algorithm 1 Rooted Beam Search

```
function RootedBeamSearch(R, S, {G_i}_{i=1}^m, β, h)        ▷ r ∈ R: root nodes,
    S = {s_1, ..., s_m}, gap constraints G_i, beam width β, heuristic h
    s_best ← ε
    B ← ∅                                                    ▷ Starting beam
    for all v ∈ R do                                         ▷ starting match extensions
        B ← B ∪ {(p^{L,v} + 1, 1)}
    end for
    V_complete ← ∅
    while B ≠ ∅ do
        Children ← ∅
        for all v ∈ B do
            if v is complete then
                V_complete ← V_complete ∪ {v}
                if v.l^v > |s_best| then
                    s^v ← Extract partial solution associated with v
                    s_best ← s^v
                end if
                continue
            end if
            C_v ← Expand(v, S, {G_i}_{i=1}^m)
            Children ← Children ∪ C_v
        end for
        Sort(Children, h)                                    ▷ decreasing order
        B ← Children[: β]
    end while
    return s_best, V_complete
end function
```

Algorithm 2 Iterative Multi-source Beam Search (IMSBS) Framework

Require: Sequences $S = \{s_1, \ldots, s_m\}$; Gap functions $G_{s_i}(\cdot), i = 1, \ldots, m$; beam width $\beta > 0$; $sources_from_R > 0$; $beam_iters > 0$, heuristics h, h', time limit t_{lim}

Ensure: Approximate common subsequence s_{best} which fulfills all gap constraints

Initialize $\mathcal{R} \leftarrow \{(\mathbf{1}, 0)\}$ ▷ A set of root nodes
for all $a \in \Sigma$ **do** ▷ Find closest matching positions for each letter
  $\mathbf{p}_a^w = (p_{i,a}^w)_{i=1}^m \leftarrow$ minimum position $p_{i,a}^w \geq 1$ and $s_i[p_{i,a}^w] = a$ (ignore all gap constraints, remove dominated root positions)
  $R \leftarrow R \cup \{(p_a^w, 0)\}$
end for
$iter \leftarrow 0$
$s_{\text{best}} \leftarrow \varepsilon$
while $\mathcal{R} \neq \emptyset \wedge iter < beam_iters \wedge t_{lim}$ not exceeded **do**
  Select $\mathcal{L} \subseteq \mathcal{R}$ $sources_from_R$ and remove them from $\mathcal{R}$ ▷ $\mathcal{R}$ prioritized acc. to h'
  for all $w \in \mathcal{L}$ **do**
    $s_{bwrd}^w, _ \leftarrow$ RootedBeamSearch($(\{(|s_i| - p_i^w)_{i=1}^m, 0\}, S^{rev}, G^{rev}, \beta', h')$ ▷ Refine each $w \in \mathcal{L}$
    $l^w \leftarrow |(s_{bwrd}^w)^{rev}| - 1$ ▷ Update l^w, assign the reverse of s_{bwrd}^w without the last letter to w
  end for
  $s_b, V_{complete} \leftarrow$ RootedBeamSearch$(\mathcal{L}, S, G, \beta, h)$
  if $|s_b| > s_{best}$ **then**
    $s_{best} \leftarrow s_b$
  end if
  for all $v \in V_{complete}$ **do**
    for all $a \in \Sigma$ **do** ▷ Find closest matching positions for each letter
      $\mathbf{p}_a^w = (p_{i,a}^w)_{i=1}^m \leftarrow$ minimum position $p_{i,a}^w \geq p_i^v$ and $s_i[p_{i,a}^w] = a$ (ignore all gap constraints)
      $\mathcal{R} \leftarrow \mathcal{R} \cup \{r^w = (\mathbf{p}_a^w, 0)\}$ (if has not previously been added) ▷ Update $\mathcal{R}$
    end for
  end for
  $iter \leftarrow iter + 1$
end while
return s_{best}

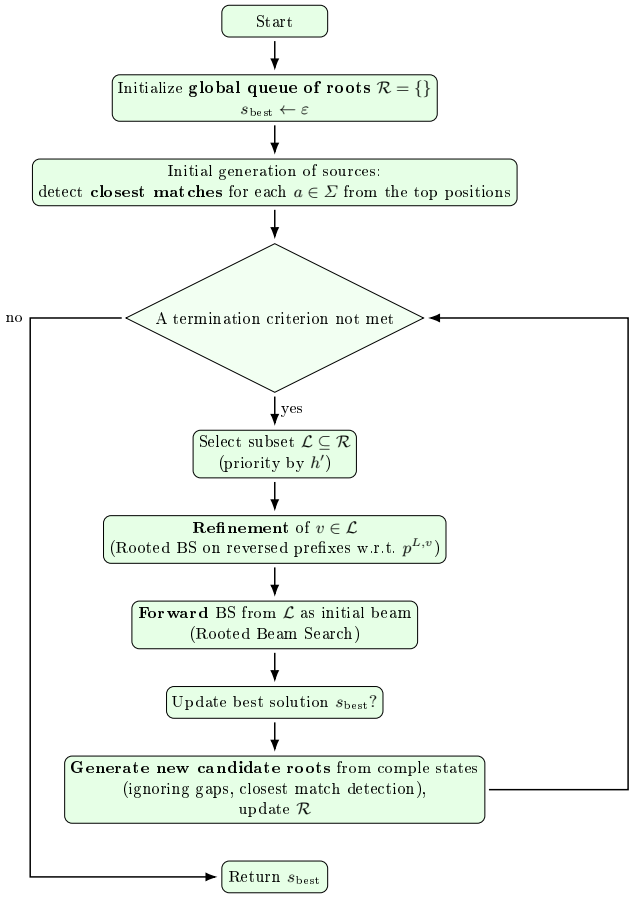
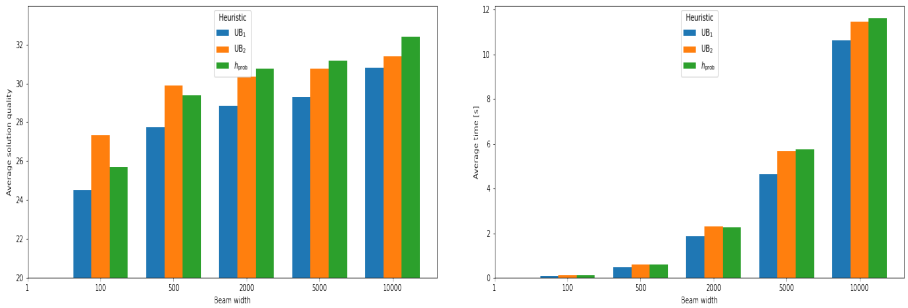


Fig. 1: Simplified workflow of the Iterative Multi-source Beam Search (IMSBS) framework.

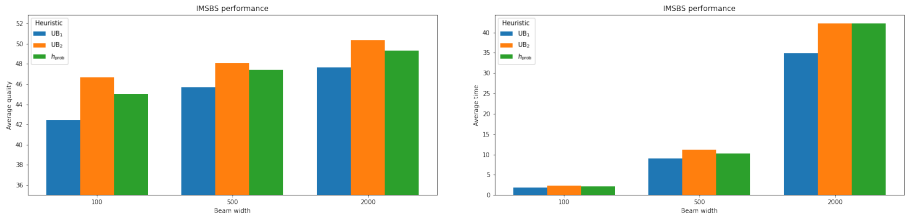
C Experimental Evaluation: Tuning & Numerical Results



(a) Avg. quality over all instances of the RANDOM benchmark suite.

(b) Avg. runtime over all instances of the RANDOM benchmark suite.

Fig. 2: Beam search tuning results on the RANDOM benchmark suite.



(a) Avg. solution quality of different IMSBS settings over all instances of the RANDOM benchmark suite.

(b) Avg. runtime of different IMSBS settings over all instances of the RANDOM benchmark suite.

Fig. 3: IMSBS parameter tuning results on the RANDOM benchmark suite.

Numerical results for all three heuristic derivatives are shown in Table 1. organized as follows. First three columns describe the instance characteristics: the number of sequences (m), the sequence length (n), and the alphabet size ($|\Sigma|$). The remaining columns report the performance metrics of three heuristic approaches: BS, IMSBS-GREEDY, and IMSBS respectively, each by its own block where two performance indicators are reported: the average objective value ($\overline{obj}$) and the average running time in seconds ($\overline{t}$), computed over 10 instances per group.

D Numerical results for the $m = 2$ case

In this section, we compare the results of approaches from the literature that are specialized for the case $m = 2$. The competitors are listed below and are described in detail in [2].

- DP: the basic dynamic programming algorithm;
- DP-1: an advanced dynamic programming approach that uses Incremental Suffix Maximum Queries (ISMQ) with `Col` and `All` matrices to accelerate the basic DP algorithm;
- DP-2: an enhanced dynamic programming algorithm that handles ISMQ using a *dequeue* data structure for further speedup. All three DP approaches are from [2]. Note that the original paper proposes answering ISMQ using Union-Find operations; however, due to a lack of implementation details, we decided to employ a dequeue-based approach in our implementation.
- ILP: an integer linear programming method proposed in this work (see Appendix A), motivated by the ILP model for the LCS problem [1].

Each algorithm was allocated 30 minutes of execution time. The ILP model was solved using the general-purpose solver CPLEX version 22.1.

The results are reported in Table 2, which is organized as follows. The first three columns describe the characteristics of the instance groups, over which the

Table 1: Numerical results on the RANDOM benchmark suite.

Inst.			BS		IMSBS-GREEDY		IMSBS	
m	n	$\|\Sigma\|$	$\overline{obj}$	$\bar{t}[s]$	$\overline{obj}$	$\bar{t}[s]$	$\overline{obj}$	$\bar{t}[s]$
2	50	2	33.6	0.02	33.1	0.00	**37.7**	0.06
2	50	4	30.1	0.98	27.7	0.00	**30.1**	0.16
2	100	2	48.9	2.07	64.5	0.01	**72.8**	0.94
2	100	4	**62.1**	11.19	56.9	0.01	61.6	0.91
2	200	2	99.1	18.56	95.5	0.02	**136.4**	6.21
2	200	4	120.5	38.15	116.1	0.05	**124.9**	6.58
2	500	2	65.3	23.27	153.6	0.07	**265.7**	119.75
2	500	4	214.6	163.69	227.7	0.12	**294.4**	60.49
3	50	2	17.5	0.03	27.2	0.00	**31.2**	0.14
3	50	4	21.7	0.18	21.5	0.00	**22.9**	0.19
3	100	2	19.7	0.06	41.8	0.01	**58.5**	3.15
3	100	4	34.1	2.35	43.4	0.03	**48.4**	5.58
3	200	2	15.2	0.16	63.6	0.02	**90.0**	22.48
3	200	4	85.3	23.45	77.2	0.08	**97.1**	72.25
3	500	2	12.6	0.10	69.9	0.07	**102.9**	53.56
3	500	4	90.7	86.35	104.2	0.29	**187.7**	412.27
5	50	2	4.8	0.00	14.9	0.00	**20.0**	0.36
5	50	4	8.9	0.01	13.6	0.06	**15.3**	0.79
5	100	2	6.3	0.01	17.7	0.01	**22.4**	0.57
5	100	4	5.3	0.01	**23.2**	10.85	22.1	1.44
5	200	2	5.3	0.01	21.6	0.03	**26.6**	1.07
5	200	4	6.4	0.02	**32.5**	604.11	25.7	2.10
5	500	2	5.9	0.10	25.5	0.14	**27.9**	3.22
5	500	4	6.8	0.10	**43.6**	1341.25	26.9	3.52
10	50	2	1.7	0.00	8.8	2.28	**9.1**	0.47
10	50	4	1.9	0.00	**7.0**	508.36	6.1	1.46
10	100	2	1.1	0.00	**14.0**	1421.10	8.6	0.54
10	100	4	2.2	0.01	**8.9**	1800.45	6.3	1.51
10	200	2	2.5	0.01	**13.2**	1710.49	10.3	0.77
10	200	4	2.2	0.02	**7.9**	1800.54	6.1	1.74
10	500	2	1.8	0.08	**13.8**	1611.70	9.5	1.53
10	500	4	1.9	0.09	**8.2**	1800.46	6.1	2.37
Avg.			32.38	10.97	46.82	394.14	**59.73**	24.63

results are averaged (10 instances per group). The remaining columns report the performance of the four algorithms: DP, DP-1, DP-2, and ILP. For each approach, both the average solution quality and the average execution time are provided across the 10 instances.

Table 2: Results on the RANDOM benchmark set for $m = 2$: the exact approaches from the literature.

Inst.			DP		DP-1		DP-2		ILP	
m	n	$\|\Sigma\|$	$\overline{obj}$	$\bar{t}[s]$	$\overline{obj}$	$\bar{t}[s]$	$\overline{obj}$	$\bar{t}[s]$	$\overline{obj}$	$\bar{t}[s]$
2	50	2	38.1	0.01	38.1	**0.01**	38.1	0.01	38.1	168.3
2	50	4	30.3	**0.01**	30.3	0.02	30.3	0.02	30.3	28.0
2	100	2	77.4	0.1	77.4	**0.03**	77.4	0.05	–	–
2	100	4	62.3	0.07	62.3	**0.06**	62.3	0.09	0.00	1800.0
2	200	2	156.4	0.75	156.4	**0.13**	156.4	0.16	–	–
2	200	4	127.2	0.59	127.2	**0.25**	127.2	0.32	–	–
2	500	2	395.9	13.57	395.9	**0.84**	395.9	1.05	–	–
2	500	4	317.2	10.18	317.2	**1.70**	317.2	2.1	–	–

Based on the numerical results reported in Table 2, the following conclusions can be drawn:

- All three dynamic programming approaches prove to be highly efficient, successfully solving all 80 problem instances to optimality.
- The runtimes required by the DP methods to obtain optimal solutions are relatively short, in most cases remaining below 10 seconds.
- The ILP approach is capable of solving only the smallest instances with $n = 50$. Its execution times are approximately two orders of magnitude higher than those of the DP approaches. For larger instances, particularly those with $n = 200$, the ILP model fails to solve even the initial (root) relaxation.

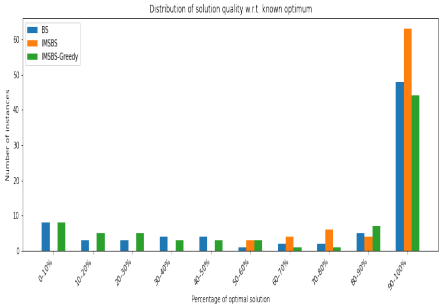
Concerning the efficiency of BS, IMSBS-GREEDY, and IMSBS on the instances solved optimally by dynamic programming ($m = 2$), Figure 4 presents the distribution of instances (y-axis) attaining specific ratios of solution quality relative to the known optimal solutions (x-axis).

In particular, IMSBS achieves near-optimal solutions in 63 out of 80 problem instances, whereas the corresponding numbers are significantly lower for the other two heuristic approaches. For instances with $|\Sigma| = 4$ (and $m = 2$), the difference in solution quality is slightly in favor of IMSBS. However, for instances with a small alphabet ($|\Sigma| = 2$), IMSBS clearly outperforms the other two approaches.

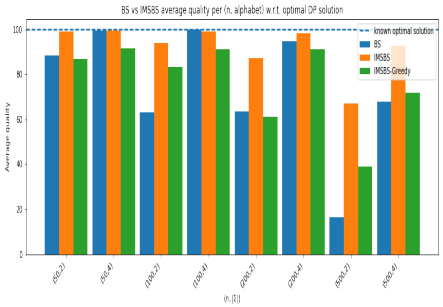
Nevertheless, for the group of instances with $n = 500$ and $|\Sigma| = 2$, there remains room for further improvement. This behavior can be explained by the known weakness of LCS-based heuristics on instances characterized by small alphabet sizes and long sequences; therefore, alternative strategies should be explored, as discussed in the future work section.

References

1. C. Blum and P. Festa, *Metaheuristics for String Problems in Bio-informatics*, Vol. 6, John Wiley & Sons, 2016.



(a) Percentage distribution of relative solution quality for each heuristic approach with respect to the optimal solutions.



(b) Relative solution quality achieved by the heuristic approaches compared to the optimal solutions.

Fig. 4: Number of instances for which Bs and Imsbs achieve a specific relative ratio of the obtained solution quality with respect to the known optimal solutions.

2. Y.-H. Penga and C.-B. Yangb, The Longest Common Subsequence Problem with Variable Gapped Constraints, In *Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan*, pp. 17–23, 2011.